AWS Lambda — Console Dashboard Wise Important Interview Questions & Answers

Lambda
│
├── Dashboard
│
├── Functions
│ └── Individual Function
│ ├── Code
│ ├── Test
│ ├── Configuration
│ │ ├── General Configuration
│ │ ├── Environment Variables
│ │ ├── Permissions
│ │ ├── Destinations
│ │ ├── Function URL
│ │ ├── VPC
│ │ ├── Monitoring & Operations Tools
│ │ └── Concurrency
│ ├── Versions
│ ├── Aliases
│ └── Layers
│
├── Applications
├── EventBridge
├── Code Signing Configurations
└── Monitor

1. What is AWS Lambda?
Answer:
AWS Lambda is a serverless compute service that runs our code without managing servers.
We upload our code and Lambda runs it when an event or request triggers the function.
AWS automatically manages the underlying infrastructure and scaling.
For example:
Suppose a user uploads an image to S3.
Lambda can automatically run a function to resize that image without us managing an EC2 server.
Example:
User Uploads Image
↓
S3
↓
Lambda
↓
Process Image
Simple ga gurthupettuko:
Lambda = Run code without managing servers

---

2. Why do we use Lambda?
Answer:
Lambda is useful when we want to run code automatically in response to events.
We don't need to continuously run or manage a server for the function.
It can automatically scale based on incoming requests.
For example:
Suppose an application receives many file uploads during the day.
Lambda can process each upload automatically whenever an S3 event occurs.
Example:
Event
↓
Lambda
↓
Run Code
↓
Result
Simple ga gurthupettuko:
Lambda = Event vachinappudu code run cheyyadam

---

3. How does AWS Lambda work?
Answer:
First, we create a Lambda function and provide the required code and configuration.
An event source or application invokes the function when the function needs to run.
Lambda creates the required execution environment, runs the code, and returns the result.

For example:
Suppose API Gateway receives an API request.
It invokes Lambda, Lambda runs our application code, and the response goes back to the client.
Example:
Client
↓
API Gateway
↓
Lambda
↓
Code Runs
↓
Response
Simple ga gurthupettuko:
Trigger → Lambda → Code → Response

Lambda Dashboard
4. What do we see in the Lambda Dashboard?
Answer:
The Lambda Dashboard provides an overview of our Lambda environment and important resources.
From the console, we can access functions, applications, monitoring, and other Lambda configuration areas.
It acts as the starting point for managing Lambda.
For example:
Suppose we want to check our existing Lambda functions.
We can open the Lambda console and go to the Functions section to view and manage them.
Example:
Lambda Dashboard
↓
Functions
↓
Select Function
↓
Manage / Monitor
Simple ga gurthupettuko:
Dashboard = Lambda starting point

Functions
5. What is a Lambda Function?
Answer:
A Lambda function is the unit of code that Lambda executes when it is invoked.
It contains our application code along with configuration such as runtime, permissions, memory, and timeout.
A function can be triggered by many AWS services and applications.
For example:
Suppose we create a Python Lambda function that processes uploaded images.
Whenever an image is uploaded to S3, the function can be invoked and execute the Python code.
Example:
S3 Event
↓
Lambda Function
↓
Python Code
↓
Process Image
Simple ga gurthupettuko:
Function = Code that Lambda executes

---

6. What is a Lambda Runtime?
Answer:
A runtime provides the environment required for Lambda to execute our function code.
It determines which programming language and supported execution environment the function uses.
Lambda supports commonly used runtimes such as Python, Node.js, Java, and others.
For example:
Suppose our development team writes the function in Python.
While creating the Lambda function, we select the appropriate Python runtime.
Example:
Lambda Function
↓
Python Runtime
↓
Python Code
↓

Execution
Simple ga gurthupettuko:
Runtime = Environment used to run the code

Code
7. What is the Code section in a Lambda function?
Answer:
The Code section is where we manage the code that the Lambda function executes.
For simple functions, we can edit the source code directly in the AWS console.
For larger applications, code is commonly packaged and deployed using other development tools or pipelines.
For example:
Suppose we have a Python function that reads an S3 object.
The Python source code is stored and deployed as part of the Lambda function.
Example:
Lambda Function
↓
Code
↓
Python / Node.js / Java
↓
Execute
Simple ga gurthupettuko:
Code = Actual application logic

Configuration
8. What is General Configuration in Lambda?
Answer:
General Configuration contains important execution settings for a Lambda function.
Common settings include memory size, timeout, and other basic function configuration.
These settings affect how the function executes and how much compute resources it receives.
For example:
Suppose our function processes a large file and takes longer than expected.
We may need to increase the function timeout and possibly allocate more memory.
Example:
Lambda
↓
General Configuration
├— Memory
└— Timeout
↓
Function Execution
Simple ga gurthupettuko:
General Configuration = Memory + Timeout + Basic settings

---

9. What is Lambda Timeout?
Answer:
Timeout defines the maximum amount of time a Lambda function can run for a single invocation.
If the function does not finish within that configured time, Lambda stops the execution.
We should set the timeout according to what the function actually needs.
For example:
Suppose a function normally finishes in 5 seconds.
We can configure a timeout that gives it enough time without allowing unnecessarily long executions.
Example:
Lambda Start
↓
Function Running
↓
Timeout Limit
↓
Finish / Stop
Simple ga gurthupettuko:
Timeout = Maximum execution time

---

10. What is Lambda Memory?
Answer:
Memory controls the amount of memory available to a Lambda function.
Lambda also allocates CPU power proportionally with the configured memory.
Increasing memory can therefore improve execution performance for some workloads.
For example:
Suppose a function is processing data slowly with a small memory configuration.
Increasing the memory may give it more CPU and allow the function to finish faster.

Example:
More Memory
↓
More CPU
↓
Potentially Faster Execution
Simple ga gurthupettuko:
Memory ↑ → CPU ↑ → Performance can improve

Environment Variables
11. What are Lambda Environment Variables?
Answer:
Environment variables store configuration values that our Lambda code can read during execution.
They help us keep configuration separate from the application code.
We can use them for values such as environment names, configuration settings, or resource identifiers.
For example:
Suppose the same Lambda code is deployed in development and production.
We can use an environment variable to tell the function which environment or resource it should use.
Example:
Lambda Code
↓
Environment Variable
↓
DEV / PROD Configuration
↓
Function
Simple ga gurthupettuko:
Environment Variable = Configuration outside code

IAM Permissions
12. What is the Lambda Execution Role?
Answer:
A Lambda execution role is an IAM role that gives the function permission to access AWS resources.
When the Lambda function runs, it uses this role to make authorized AWS API calls.
The role should contain only the permissions required by the function.
For example:
Suppose a Lambda function needs to read objects from S3.
We attach an execution role with the required S3 read permission to the function.
Example:
Lambda
↓
Execution Role
↓
IAM Permissions
↓
S3
Simple ga gurthupettuko:
Execution Role = What Lambda is allowed to access

Triggers
13. What is a Lambda Trigger?
Answer:
A trigger is an event or service that invokes a Lambda function.
Many AWS services can invoke Lambda when a specific event occurs.
Common examples include API Gateway, S3, EventBridge, SQS, and other AWS services.
For example:
Suppose an object is uploaded to S3.
The S3 event can trigger Lambda, and Lambda can automatically process the uploaded object.
Example:
S3 Upload
↓
Trigger
↓
Lambda
↓
Process Object
Simple ga gurthupettuko:
Trigger = Event that starts Lambda

---

14. What is the difference between Synchronous and Asynchronous Lambda invocation?

Answer:
With synchronous invocation, the caller waits for Lambda to execute and return a response.
With asynchronous invocation, the event is accepted and Lambda processes it without the caller waiting for the final result.
The choice depends on how the application needs to handle the response.
For example:
An API request usually needs a response immediately, so synchronous invocation is common.
An S3 event can be processed asynchronously because the uploader does not need to wait for the Lambda result.
Example:
Synchronous:
Client → Lambda → Response

Asynchronous:
Event → Lambda → Process
Simple ga gurthupettuko:
Sync = Wait for response
Async = Don't wait

VPC
15. Can Lambda run inside a VPC?
Answer:
Yes, a Lambda function can be configured to connect to a VPC.
This is useful when the function needs private network access to resources such as private RDS databases.
We configure the required VPC subnets and security groups for the function.
For example:
Suppose our RDS database is private inside a VPC.
We can configure Lambda with appropriate VPC networking so the function can connect to that database.
Example:
Lambda
↓
VPC
↓
Private Subnet
↓
RDS
Simple ga gurthupettuko:
Lambda + VPC = Access private resources

---

16. Why would we connect Lambda to a VPC?
Answer:
We connect Lambda to a VPC when the function needs to access resources that are reachable through private networking.
Typical examples include private databases, internal services, or other VPC resources.
The VPC configuration must include suitable subnets and security groups.
For example:
Suppose an application uses a private RDS database that is not publicly accessible.
A Lambda function can be connected to the VPC so it can communicate with the database privately.
Example:
Lambda
↓
VPC
↓
Private Network
↓
Private RDS
Simple ga gurthupettuko:
VPC Lambda = Private network access

Concurrency
17. What is Lambda Concurrency?
Answer:
Concurrency represents the number of Lambda function executions running at the same time.
Lambda can automatically create multiple execution environments when many requests arrive.
Concurrency controls are useful when we need to manage scaling and protect downstream systems.
For example:
Suppose 100 requests arrive at a Lambda function at the same time.
Lambda can run multiple executions concurrently instead of processing all requests one by one.
Example:
100 Requests
↓
Lambda
↓
Multiple Concurrent Executions

↓
Process Requests
Simple ga gurthupettuko:
Concurrency = How many executions run at once

---

18. What is Reserved Concurrency?
Answer:
Reserved concurrency sets a specific concurrency limit for a Lambda function.
It also reserves that amount of concurrency for the function so other functions cannot use it.
This is useful when we want to control Lambda scaling or protect downstream resources.
For example:
Suppose our database can safely handle only a limited number of Lambda connections.
We can configure reserved concurrency to prevent the Lambda function from creating too many simultaneous executions.
Example:
Lambda
↓
Reserved Concurrency
↓
Maximum Controlled Executions
↓
Database Protected
Simple ga gurthupettuko:
Reserved Concurrency = Control Lambda execution limit

Versions & Aliases
19. What are Lambda Versions?
Answer:
Lambda Versions allow us to keep different published versions of a function's code and configuration.
A published version is immutable, so we can safely keep an older working version.
This is useful for deployments and rollback.
For example:
Suppose version 1 is working correctly and we deploy version 2.
If version 2 has a problem, we can use the previous version for rollback.
Example:
Lambda
├── Version 1
├── Version 2
└── Version 3
Simple ga gurthupettuko:
Version = Saved Lambda release

---

20. What is a Lambda Alias?
Answer:
An alias is a name that points to a specific Lambda function version.
Instead of applications directly using version numbers, they can use aliases such as dev, test, or prod.
Aliases make deployments and version management easier.
For example:
Suppose prod currently points to version 5.
After testing version 6, we can move the prod alias to version 6.
Example:
prod
↓
Lambda Alias
↓
Version 5

Later:
prod
↓
Version 6
Simple ga gurthupettuko:
Alias = Friendly name pointing to a version

Layers
21. What is a Lambda Layer?
Answer:
A Lambda Layer is a package containing reusable code, libraries, or dependencies that Lambda functions can use.
It helps us avoid packaging the same dependencies separately with every function.
Multiple Lambda functions can use the same layer.

For example:
Suppose ten Python Lambda functions need the same common library.
Instead of packaging that library into every function, we can put it in a Lambda Layer and reuse it.
Example:
Lambda 1 ─┐
Lambda 2 ─┼─> Common Layer
Lambda 3 ─┘
Simple ga gurthupettuko:
Layer = Reusable dependencies

Function URL
22. What is a Lambda Function URL?
Answer:
A Lambda Function URL provides an HTTPS endpoint directly for invoking a Lambda function.
It can be useful when we need a simple HTTP endpoint without putting API Gateway in front of the function.
Access can be controlled using the configured authorization settings.
For example:
Suppose we have a simple HTTP-based Lambda function.
Instead of creating API Gateway, we can use a Function URL to provide an HTTPS endpoint for the function.
Example:
Client
↓
HTTPS Function URL
↓
Lambda
↓
Response
Simple ga gurthupettuko:
Function URL = Direct HTTPS endpoint for Lambda

Destinations
23. What are Lambda Destinations?
Answer:
Lambda Destinations allow us to send the result of an asynchronous Lambda invocation to another service.
We can configure different destinations for successful or failed executions.
This makes it easier to build workflows around Lambda results.
For example:
Suppose a Lambda function processes an order asynchronously.
After successful processing, we can send the result to another service, while failed executions can be sent to a failure destination.
Example:
Event
↓
Lambda
├─ Success → Destination
└─ Failure → Destination
Simple ga gurthupettuko:
Destinations = Send Lambda result somewhere

Monitoring
24. How do we monitor a Lambda function?
Answer:
Lambda integrates with Amazon CloudWatch for monitoring function performance and execution logs.
We can check metrics such as invocations, errors, duration, and throttles.
CloudWatch Logs can help us troubleshoot application output and errors.
For example:
Suppose a Lambda function suddenly starts failing.
We can check CloudWatch metrics and logs to understand the error and investigate the problem.
Example:
Lambda
↓
CloudWatch
├─ Metrics
└─ Logs
↓
Troubleshooting
Simple ga gurthupettuko:
Lambda + CloudWatch = Monitor + Troubleshoot

Applications
25. What is a Lambda Application?

Answer:
A Lambda application is a collection of related AWS resources used together to implement an application or serverless solution.
It can include Lambda functions and other supporting resources.
This helps organize serverless applications as a complete solution rather than as isolated functions.
For example:
Suppose an image-processing application uses S3, Lambda, and other AWS resources.
These related components can be managed as one serverless application.
Example:
Serverless Application
├— S3
├— Lambda
└— Other AWS Resources
Simple ga gurthupettuko:
Application = Related serverless resources together

Important Interview Scenarios
26. Lambda vs EC2 — what is the difference?
Answer:
EC2 gives us a virtual server where we manage the operating system and server configuration.
Lambda is serverless, so AWS manages the underlying servers and we mainly manage our code and configuration.
Lambda is commonly used for event-driven workloads, while EC2 is useful when we need more control over the server.
For example:
If we need a continuously running custom server, EC2 may be suitable.
If we only need code to run when an event occurs, Lambda can be a better choice.
Example:
EC2
↓
Virtual Server
↓
OS + Application

Lambda
↓
Code
↓
AWS Manages Servers
Simple ga gurthupettuko:
EC2 = Manage server
Lambda = Manage code

---

27. What happens when Lambda receives many requests?
Answer:
Lambda can create additional execution environments to handle multiple requests concurrently.
This allows the function to scale automatically based on incoming workload.
However, concurrency limits and downstream service capacity should be considered.
For example:
Suppose an API normally receives 10 requests but suddenly receives thousands.
Lambda can scale to handle concurrent executions, but we should ensure databases and other dependencies can handle the increased traffic.
Example:
Many Requests
↓
Lambda
↓
Multiple Execution Environments
↓
Concurrent Processing
Simple ga gurthupettuko:
More requests → More Lambda executions

---

28. What are common reasons a Lambda function fails?
Answer:
A Lambda function can fail because of application code errors, timeout, insufficient permissions, incorrect configuration, or network problems.
If it is connected to a VPC, incorrect subnet or security-group configuration can also cause connectivity problems.
CloudWatch Logs and metrics are commonly used to troubleshoot failures.
For example:
Suppose Lambda needs to read from S3 but its execution role does not have permission.
The function can fail because the required AWS API call is not authorized.
Example:
Lambda Failure

├— Code Error
├— Timeout
├— IAM Permission
├— Network
└— Configuration

Simple ga gurthupettuko:
Lambda issue → Check Code + IAM + Timeout + Network

---

29. How would you deploy a new Lambda version safely?
Answer:
I would publish a new Lambda version instead of directly replacing the working version.
Then I can use an alias such as prod to point applications to the required version.
This makes rollback easier if the new version has a problem.
For example:
Suppose production is running version 10 and we release version 11.
We test version 11 first and then move the production alias to version 11.
Example:
Production
↓
prod Alias
↓
Version 10

After Testing
↓
prod Alias
↓
Version 11
Simple ga gurthupettuko:
Version + Alias = Safer deployment & rollback

---

30. Explain Lambda in one real-world example.
Answer:
Suppose users upload images to an S3 bucket in our application.
The S3 event triggers a Lambda function, and Lambda processes the uploaded image without requiring a dedicated server.
CloudWatch monitors the function, while IAM permissions control what the Lambda function can access.
Example:
User
↓
Upload Image
↓
S3 Bucket
↓
S3 Event
↓
Lambda
↓
Process / Resize Image
↓
S3
Simple ga gurthupettuko:
S3 Event → Lambda → Process → Store Result